

Usage, Effects and Requirements for AI Coding Assistants in the Enterprise: An Empirical Study

Michele Merler
IBM Research
Yorktown Heights, NY, USA
mimerler@us.ibm.com

Tin Kam Ho
IBM Research
Yorktown Heights, NY, USA
tho@us.ibm.com

Rangeet Pan
IBM Research
Yorktown Heights, NY, USA
rangeet.pan@ibm.com

Raju Pavuluri
IBM Research
Yorktown Heights, NY, USA
pavuluri@us.ibm.com

Rahul Krishna
IBM Research
Yorktown Heights, NY, USA
rkrsn@ibm.com

Maja Vukovic*
IBM Research
Yorktown Heights, NY, USA
maja@us.ibm.com

Abstract

The rise of large language models (LLMs) has accelerated the development of automated techniques and tools for supporting various software engineering tasks, e.g., program understanding, code generation, software testing, and program repair. As CodeLLMs are being employed toward automating these tasks, one question that arises, especially in enterprise settings, is whether these coding assistants and the code LLMs that power them are ready for real-world projects and enterprise use cases, and how do they impact the existing software engineering process and user experience. In this paper we survey 57 developers from different domains and with varying software engineering skill about their experience with AI coding assistants and CodeLLMs. We also reviewed 35 user surveys on the usage, experience and expectations of professionals and students using AI coding assistants and CodeLLMs. Based on our study findings and analysis of existing surveys, we discuss the requirements for AI-powered coding assistants.

CCS Concepts

• **Human-centered computing** → **Empirical studies in HCI**.

Keywords

AI Coding Assistant, Survey, Usability Study, CodeLLM

ACM Reference Format:

Michele Merler, Rangeet Pan, Rahul Krishna, Tin Kam Ho, Raju Pavuluri, and Maja Vukovic. 2026. Usage, Effects and Requirements for AI Coding Assistants in the Enterprise: An Empirical Study. In *3rd International Workshop on Large Language Models For Code (LLM4Code '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3786181.3788727>

1 Introduction

The evolution of modern software engineering has consistently leaned toward automation so as to increase efficiency and reduce repetitive and low-level development tasks. Over the past decade,

various techniques ranging from the evolution of scripting utilities to more sophisticated development frameworks have attempted to simplify common software development tasks [14]. The advent of Code Large Language Models (CodeLLM) powered AI-coding assistants represents yet another promising step in this direction, allowing developers to automate code generation, suggest completions, produce tests, and even modernize legacy codebases. The use of large language models (LLMs) allows these AI assistants to enable contextually relevant guidance, streamline code creation, and offer assistance in learning new programming languages. More recently, the rise of agentic systems has enabled further automation of more complex software engineering tasks such as bug fixing [18].

In this paper we survey 57 industry developers about their experience with AI coding assistants. The surveyed group came from several functional domains (e.g. software, consulting, research, etc.), with varying levels of software engineering experience. The intent of this survey was to understand the following:

- What are the different utilization of coding assistants and desired use cases across the functional verticals
- How does the development experience influence the expectations of an AI coding assistant.
- What are the differences in software engineering processes introduced by AI coding assistants.
- What is the overall perceived effectiveness of the assistants
- What are the requirements for the future AI coding assistants.

Stemming from the user study, we look for requirements that will shape the research road map and features for future iterations of coding assistants. The contribution of our work is a survey that provides a comparative study of usage of AI coding assistants across a diverse set of users, understanding the impact of that usage in terms of organizational structure, productivity and quality.

2 Related Work

Research on AI coding assistants has grown rapidly, with GitHub Copilot, ChatGPT, Tabnine, and enterprise specific tools such as Amazon Q and IBM Watsonx Code Assistant among the most studied [36]. Large surveys [42, 44, 52, 54] and randomized controlled trials [7, 10, 27, 37] consistently show perceived productivity gains [51], often in the range of 12–25% speedups and up to almost one third of developers code being written with help from the AI tool



This work is licensed under a Creative Commons Attribution 4.0 International License. *LLM4Code '26, Rio de Janeiro, Brazil*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2412-1/26/04
<https://doi.org/10.1145/3786181.3788727>

[25]. Industry case studies further validate these benefits in real-world settings [3, 46]. Major categories with satisfactory outcomes include code generation and refactoring, testing, code explanation and addressing data analytics issues [12].

While productivity advantages are clear, the extent of improvement varies across tasks type and complexity [50], developer expertise [16], and organizational contexts. Less experienced and causal developers report higher adoption rates of AI coding tools and greater productivity gains [11], using them to reduce opportunity costs. In open source, evidence has been found that this enhanced development activity translated even into increased short to medium-term career benefits for developers [23]. On the other hand recent studies [5] have found discrepancies between perceived and measured productivity for experienced developers.

Beyond efficiency, concerns about quality and security remain salient. Empirical audits show that Copilot-generated code often contains vulnerabilities, particularly in security-critical domains [15, 20, 39, 48]. Perceptions are similarly mixed: developers report faster delivery but raise doubts about maintainability, integration and correctness [24, 29, 49]. Studies on readability and naming practices find no consistent degradation but highlight variability depending on context [22, 46, 53]. Overall, the literature underscores that productivity gains may come at the cost of quality assurance and over-reliance is identified as a risk factor [1].

Adoption and trust [35, 41] have emerged as central challenges. Legal and ethical questions compound these issues, as many practitioners remain unclear about copyright and licensing implications [4, 44]. Recent studies [31] found that widespread organizational expectations in industries for rapid productivity gains are limited by uneven adoption and familiarity with the tools from different team members, thus advocating for needed investment in learning. Studies of learning and collaboration reveal similarly dual effects [26]. Novices benefit from scaffolding and improved comprehension of large codebases [6, 28, 40, 43], yet risk shallow learning and over-reliance [1, 2]. Professionals likewise report efficiency gains but worry about deskilling in core competencies, even as new skills around prompting and verification emerge [9, 38, 45].

The socio-economical consequences of the wide adoption of AI based coding tools are also being researched, with focus not only on workflow efficiency but also its potential for developer replacement [21] and displacement [13]. The use of AI coding assistants beyond software engineers has been largely understudied, with few exceptions discovering for example that scientists often use code generating models as an information retrieval tool for navigating unfamiliar programming languages and libraries [34].

Despite these advances, our analysis of prior studies reveals significant gaps. First, most of the studies focus on immediate productivity or single specific tasks [8, 32, 33] rather than long-term maintainability, software quality, or organizational outcomes. Second, despite the recent proliferation of AI tools (Cursor, Windsurf, Replit, Zencoder, Cody, BlackboxAI, abacus AI, Qodo, Bolt etc.) studies have predominantly focused only on very few, namely ChatGPT and Github Copilot. Third, participants are often drawn from homogeneous cohorts, restricting generalizability across industries, regions, and developer demographics. These omissions highlight the need for more diverse, longitudinal, and governance-focused studies to inform both research and practice.

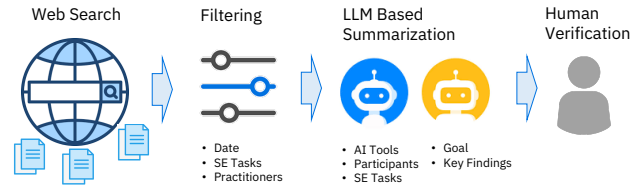


Figure 1: Surveys search, selection and analysis process.

3 Survey on AI coding assistants use in enterprise and academia

In this Section we review 35 user surveys on the usage, experience and expectations of professionals and students using AI coding assistants and codeLLMs. While the focus of our work is on industry use-cases, we expanded the scope of the reviewed surveys to include academia as a comparison point. In the following we report the insights from this review, starting with the description of the methods used to search and collect the papers, the filtering criteria, the breakdown of AI tools and models investigated, the software engineering tasks involved, the participants information and goal of the surveys. Finally we report the collective key insights. A summary of the process is illustrated in Figure 1.

3.1 Search Selection Criteria

We started from a web search using terms such as “survey”, “study”, “user experience”, “insights” in correlation with “AI coding assistant”. From the initial pool of candidate papers, we further expanded the search to any works citing those papers. The selection criteria for the works to include in this review were then the following.

Recent. As the landscape of the features and capabilities of AI coding assistants and the models powering them is progressing at staggering speed, we decided to include only studies from the past two years (2024 and 2025), with two exceptions from 2023 covering the initial opinions of SE using AI coding assistants.

Practitioners Surveys. We excluded works that contained empirical studies analyzing the features and capabilities of AI coding assistants, but did not include human surveys. Our goal was to understand what has been asked of practitioners using AI coding assistants and which conclusions can be drawn from it.

Specifically Covering SE Tasks. Our goal is to focus on the perceptions, usage and vision for AI coding assistants specifically for software engineering. Some works were excluded since they surveyed practitioners on other aspects of AI coding assistants.

Publicly Available. We included works that were accessible through Google Scholar, ArXiv and conferences and journal sites. We excluded enterprise works not publicly disclosed (the only report from a private consulting company [30] was publicly available).

From an initial pool of approximately 50 papers we retained 35 published between 2023 (2), 2024 (14) and 2025 (19).

3.2 Methodology

For each survey, we designed to compare and summarize the following information: which AI Tool(s) it investigated, the number and participants and their background (role, expertise, industry, proficiency levels), the software engineering tasks covered, the overall goal of the study and finally the key findings. In order to

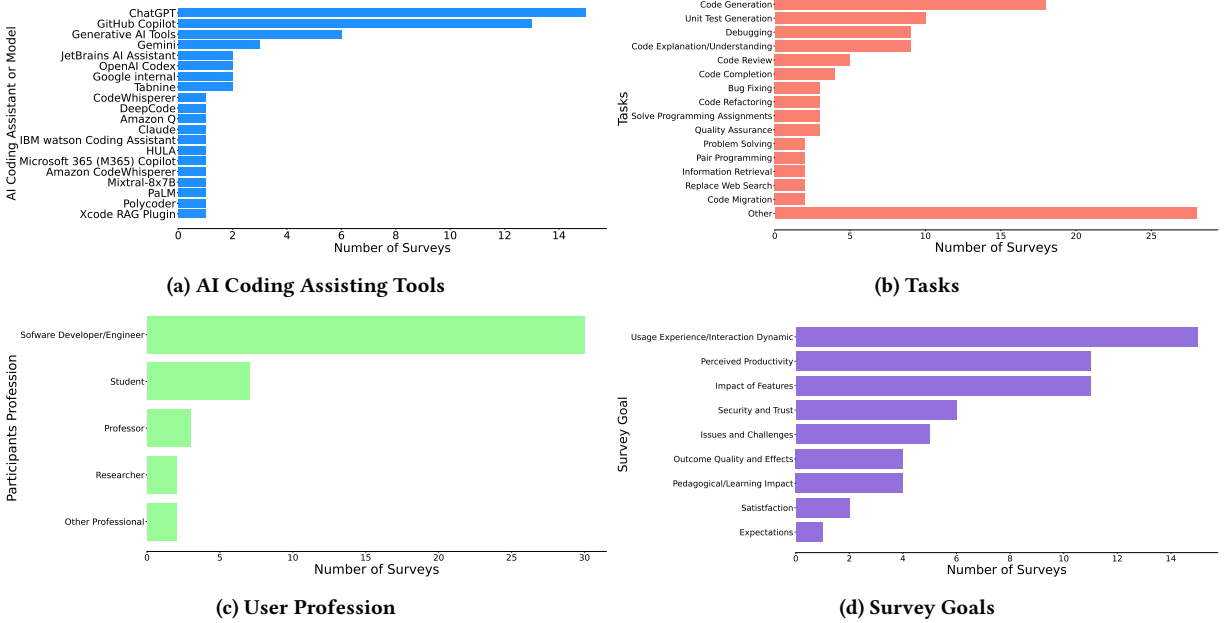


Figure 2: Distribution of AI Tools (a), Tasks (b) Users Professions (c) and Goals (d) covered in the 35 SE surveys analyzed.

facilitate the processing of relevant information from the pool of selected papers, we used two frontier models to analyze them at scale, namely Gemini 2.5 pro and Claude Sonnet 4. We prompted each model to act as a research scientist and provide the best prompt for itself to extract the information we designed from each of the papers (details can be found in the Supplementary Material).

We then proceeded to iteratively prompt each model for each of the papers selected. The information for each paper was then manually verified and summarized (details can be found in the Supplementary Material). We then performed an aggregated analysis on the extracted information, resulting in the following conclusions.

3.3 Analysis, Gaps and Improvement Areas

From the analysis of the surveys outcomes, productivity gains has been identified as the number one perceived benefit. This stems from being able to automate and/or accelerate tedious repetitive tasks as well as simple modifications to existing code. Rapid prototyping and learning opportunities emerge also as clear perceived benefits of software solutions.

While the large (and growing) quantity of user studies is providing useful insights in usage and perceived utility of AI coding assistants, our analysis of prior studies reveals significant gaps. First, the empirical base is dominated by ChatGPT and GitHub Copilot (as clearly evident by the distribution in Figure 2a), leaving comparative understanding of other assistants limited.

Second, most of the studies we found are short-term, focusing on immediate productivity or single specific tasks rather than long-term maintainability, software quality, organizational outcomes or desired new features. Only one study surveyed developers expectations as opposed to experience and outcomes (as seen in the Goals breakdown in Figure 2d). Looking at the distribution of tasks reported in Figure 2b, we can observe how Code Generation, Unit

Test Generation, Debugging and Code Explanation/Understanding dominate. Quality assurance is part of only four surveys. This could be a result of both the features offered/advertised so far by AI coding Assistants as well as the limitations of the surveys in asking desiderata from the participants, as opposed to experience with existing functionalities. The long tail of “Other” tasks mentioned in only one survey includes: Diff Verification, Conditional Logic, Commits Writing, Code Selection, Issue Ideation, Expedite Manual Work, Functionality Verification, Implementing New Features, Next-step Hints Generation, Jump-start First Draft, Issue Synthesis, Note Taking, Accelerate Updates, Advanced Reasoning, Code Readability, Reduced Development Time, Pull Requests, Program Repair, Security-related Programming Tasks, Smart Paste, Tackle New Challenges, Testing, Threat Modeling, Trustworthiness Assessment, Workflow Automation, and General Writing Assistance.

Third, research participants are drawn from homogeneous cohorts, limiting generalization across industries, regions, and demographics. Though studies showed high variability in participant numbers (10 to 17,420) and experience levels (novice to 15+ years), few selected cohorts across different industries or geographies. Very few studies examined specialized groups beyond software engineers and developers: six focused on students using AI tools for learning coding and solving programming puzzles, and two included researchers, as shown in Figure 2c. None compared the use and desired features of AI coding assistants across groups even within the same company, but working on different divisions and roles. We intend to fill this gap with Research Question 1 in our own survey in Section 4.3.

Fourth, there is a notable absence of analysis targeted to agentic workflows. Given the recent emergence of agentic-first coding assistants offered as specific IDEs or plugins (Cursor, RooCode, Windsurf, Replit, etc.) and CLI options (Gemini cli, Claude cli, OpenAI

codex, etc.) the lack of analysis of users experience and expectations specifically around agentic capabilities is a gap that needs to be filled. We call on the SE Research community to fill such gap.

Finally, while trust, governance, and ethical risks are widely acknowledged in the literature [17, 19, 47], systematic evaluation of solutions—such as personalization, explainability, or policy frameworks remains scarce. These omissions highlight the need for more diverse, longitudinal, and governance-focused studies to inform both research and practice.

4 Survey

4.1 Questionnaire

We developed a comprehensive survey, with 25 questions, capturing a wide range of data regarding not only the user experience with AI coding assistants, but also practitioners expectations on existing and potentially new features that such assistants can provide, specifically tailored to their workstreams. We launched data collection in May of 2025 within a tech company, and received 57 responses. Data was collected through an on-line form.

4.2 Participant Demographics

In this section, we discuss the demographics of the participants. In total, we received 57 responses from a significantly diverse population. Figures 3a, 3b, and 3c illustrate the key characteristics of the participant pool. Overall, the participants represent a range of departments, including Finance and Operations (30%), Software (25%), and Research (25%), among others. We also found that the participants are highly experienced, with 42% having more than 20 years of experience. At the same time, the pool includes new hires, with 26% reporting less than 5 years of experience. Regarding programming language background, while most participants reported using AI assistants for Python, Java, and JavaScript, a significant portion also reported using coding assistants for other languages such as C, Go, Rust, COBOL, C++, C#, and others.

4.3 Research Questions

We designed several questions to assess not only the overall perceived effectiveness of AI coding assistants, but also desired features and unique uses across divisions.

- (1) What was the motivation for you to use the AI coding assistant?
- (2) Which type of help do you benefit the most from it?
- (3) What fraction of the AI generated code do you actually keep/use?
- (4) Is the use of AI coding assistant a productivity gain for you?
- (5) How did your software engineering process/development change when you started using an AI coding assistant?
- (6) What are the features of a future ideal AI coding assistant?

4.3.1 Motivation to use AI coding assistant. Users reported various motivations to use an AI based coding assistant. While some were driven by curiosity about the new technology, most had expectations that it can improve their productivity and shorten development time. These included work in several phases of development, such as exploring ideas for some new program design, fast prototyping, or speeding up an ongoing project, especially when there is a need for much boilerplate code, common helper functions, or highly repetitive editing and transformation. Others hoped to get help in

understanding other people’s code, or generating documentation for their existing code. Some expected to save effort in generating tests, improving their code, refactoring, and finding solutions to issues. Interestingly, several users mentioned that they wanted advice on best practices in coding, or leveraging the assistant to learn about a new package or an unfamiliar programming language.

4.3.2 Differences across divisions in motivation, use and desired features. Users from different divisions cited multiple motivations for using an AI coding assistant. We manually mapped the free-text motivations written by users to a set of six general themes: Code Quality Improvement, Productivity/Efficiency, Creative Collaboration, Understanding/Documenting, Curiosity and Other (which includes all answers that could not be mapped to any of the previous themes). While the majority of users in every division employs AI coding assistants for efficiency and productivity gains, from the data reported in Figure 6 we can observe substantial differences across divisions in the distribution of motivations. Within the Research division, productivity is basically the sole motivating use case, while in Global Sales it holds the same importance as improving code quality. Within the Consulting division the understanding code is also a very prominent motivation, whereas in Software we observe a multitude of themes being almost equally cited. It also is interesting to see very specific examples that were reported in the Other category: for example “synthetic data generation” in Research or help to “sell a product” in Global Sales.

In terms of perceived value provided by the different features offered by AI coding assistants, we observe large variations in rankings across divisions, as shown in Figure 7. While everyone values *correctness* above all, *security* is the most valuable contribution for Sales, while Research values more the ability to *customize* to very specific codebases and domains, Quantum being a domain mentioned multiple times.

Given the large variety of motivating use cases and adoption types, tailor modes within AI coding assistant are desirable, optimized and targeted for specific use cases.

Motivating Usecases Takeaway

- (1) There is no “one size fits all” motivating use case and adoption type across users of AI coding assistants.
- (2) Tailor modes/models within AI coding assistants are desirable, optimized and targeted for specific use cases.

4.3.3 Productivity Gains and Code Retention. The vast majority of respondents found adapting to the use of AI coding assistants easy (39% somewhat easy, 47% extremely easy). 88% of them perceived productivity gains when using AI Coding assistant. Among the 57 responding users, 44 of them reported at least a 25% increase in productivity. A smaller fraction (26 users) reported at least a 50% increase in productivity. 18% estimate up to 100% process improvement. We then went on to understand what, if any traditional, non-AI technologies, users have replaced by AI coding assistant. Over 65% respondents reported that they previously relied on Google Search, followed by StackOverflow for bug fixes (whether code, config, etc.), and nonAI IDE completion before transitioning to an AI coding assistant. We then went on to understand user’s daily tasks and requirements of a AI coding assistant specific to their role. Out of 35 respondents to this question, 47% reported that their

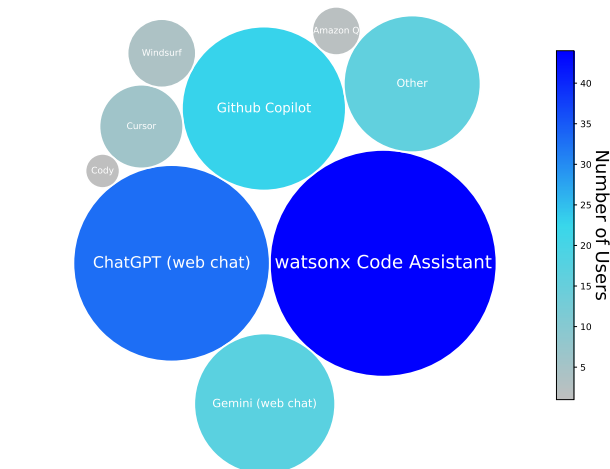
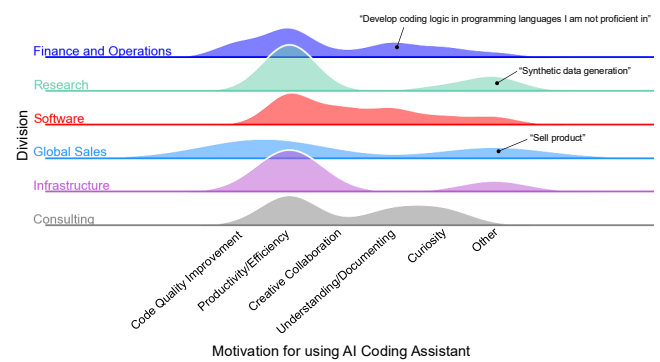
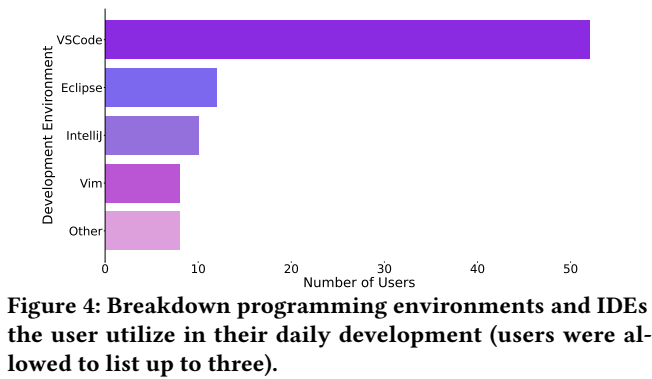
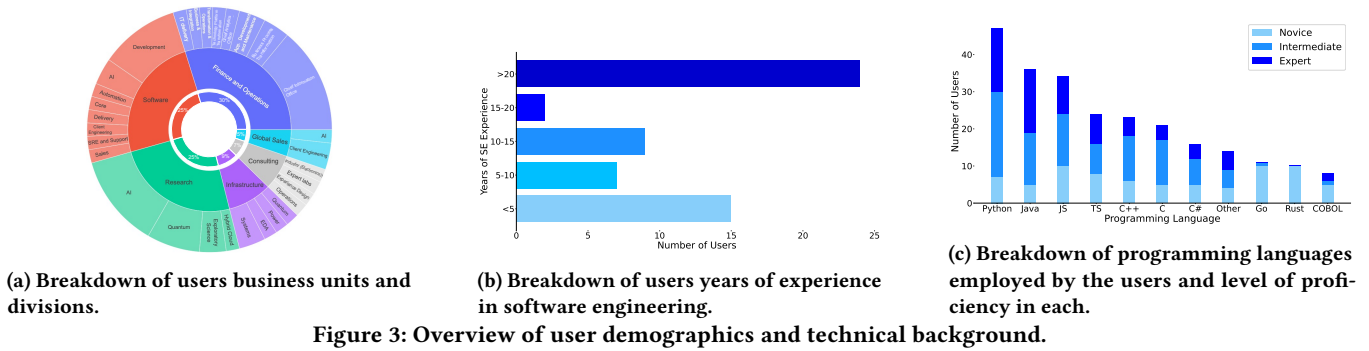


Figure 5: Distribution of AI coding assistants with which the users have had hands on experience in the past (users were allowed to list up to three).

focus is on code generation (including guidance for coding, code reviews), followed by code understanding and testing.

The majority of users (48) kept the code generated by the AI coding assistant. A large portion of users retained between 25-50% of the code generated. Many users reporting productivity gains used the AI coding assistant daily or weekly.

Figure 6: Distribution of reasons for using AI coding assistants across divisions.

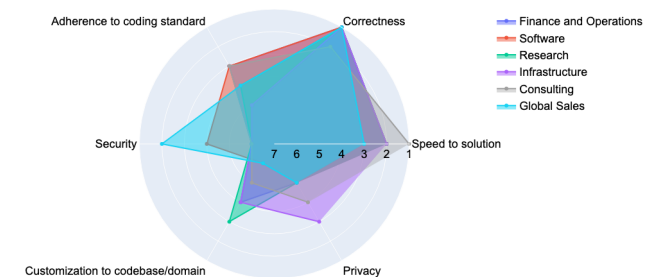


Figure 7: Ranking of perceived value of features offered by AI coding assistants, presented by division. Rank of 1 signifies the most valuable feature.

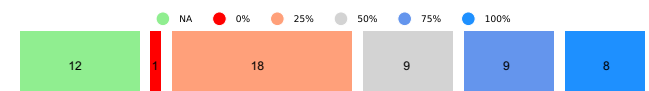


Figure 8: Distribution of Estimated Productivity. The colors represent perceived productivity gain from 0% (red) to 100% (blue). 44 out of 57 users (97.8% of the users who answered on the perceived gain) reported a gain of at least 25%.

Benefit and Tool Replacement Takeaway

Majority of users experience productivity gains from AI coding assistants, primarily in code generation and code understanding; replacing their prior usage of Web tools such as StackOverflow or Google search.

4.3.4 Help received from the AI coding assistant. Users reporting productivity gains found benefits in both learning and implementation processes. On the learning side, AI provided knowledge about available packages or coding methods, as well as quick access to available documentation and relevant information; AI also helped in putting together documentation of their code for others to understand. On the implementation side, AI helped speeding up the development process, from providing suggestions for coding logic, prototyping and scaffolding solutions, generating code drafts, to finishing the full solution via testing and debugging.

4.3.5 Change in software engineering process due to using an AI coding assistant. According to the responders, using an AI code assistant has significantly improved development efficiency and quality. It has become an essential tool for many, helping to (a) follow coding standards and best practices to write clearer code, (b) save time on repetitive tasks and prototyping, (c) quickly identify issues and errors in debugging, (d) ensure compliance with security standards and (e) better understand existing projects and facilitate maintaining and improving them. A majority of the surveyed practitioners, 42 out of 57 (73.7%), reported that the use of AI code assistants has transformed their development process. Specifically it has enabled them to (a) deliver required features and solutions more quickly, (b) focus on high-level design and decision-making, (c) reduce work on repetitive tasks thus saving time for more complex problems and finally (d) try out new technologies and languages with less effort. The use of AI code assistants varies depending on the context, with some developers using them extensively in personal projects while others are restricted by company policies.

4.3.6 Reasons for those users perceiving no gain from using an AI coding assistant. According to the smaller fraction (13 of 57 (22%)) of responding users who reported little or no gain in productivity resulting from the use of an AI coding assistant, the reasons are:

- Limitations of the coding assistant: AI can generate boilerplate code but not those that need custom or complex logic.
- Correctness and completeness: the code generated by AI may contain errors, omit important features, or not follow standard practices in the codebase.
- Prompting needs: a detailed prompt is needed to fully specify the requirements and obtain accurate and relevant code, which is not easier than writing the code manually instead.

As a result, they saw negative impact on development, i.e., using AI tools has actually made it harder to debug and fix issues.

A few developers used languages in their current projects that are not supported by the AI coding assistant. Some preferred to use other IDEs not integrated with an AI coding assistant, and were therefore not able to obtain the expected benefits.

Perceived Helpfulness Takeaway

AI coding assistants are perceived as helpful to

- (1) deliver required features and solutions more quickly.
- (2) focus on high-level design and decision-making.
- (3) reduce work on repetitive tasks.
- (4) try out new technologies and languages more easily.

4.3.7 Features of an ideal AI coding assistant. We asked users to list desirable features not yet offered by current AI coding assistants,

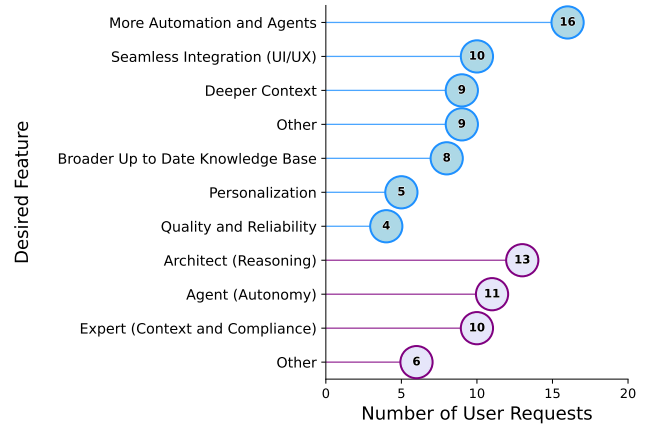


Figure 9: Feature requests for ideal AI coding assistants. Short-term (incremental, top in blue) vs Long-term (transformative, bottom in purple.)

distinguishing between short term, that is, perceived as feasible in the near future and more incremental in nature with respect to existing capabilities, and longer term, more transformative directions. The results are reported in Figure 9.

Short-term features. We mapped the respondents' requests into the following main proposals for incremental improvements.

More automation and agents. The most frequent request is for the assistant to have high quality support for Git (PRs/merges), CI/CD integration, and issue management, proactive bug detection and automated fixes. Also, interactivity with the user to establish a form of pair programming routine.

Deeper context. Users want the assistant to have deep, full-repository awareness. It must be able to scan and understand their entire local codebase, project files, and documentation without RAG database setup or performance degradation after a certain token count.

Seamless Integration (UI/UX). Users want the assistant to be a part of their core workflow, not a separate tool. This means full integration with their development environment to enable direct, in-line code edits and quick fixes for all tasks.

Broader and up to date knowledge base. The assistant's knowledge is seen as too limited. Users demand support for more specific, less popular languages (specifically naming Perl, Shell, Apex, OPL) and up-to-date knowledge of modern SDKs, libraries, and frameworks, including highly functional library version management with monitoring of incompatibilities.

Quality and reliability. Improve core quality by being faster and more accurate. A key request is for the assistant to state its confidence level and manifest uncertainty when appropriate, rather than being "confidently incorrect".

Personalization. Allow users to choose their model (e.g., local vs. remote) and, most importantly, have the assistant learn and adapt to their personal coding style.

Among other suggestions were also to (a) evolve testing capabilities from just unit tests to generating API and UI tests and (b) enable multimodal interaction modes, such as voice based.

Long-term features. Shifting the focus from the "practical present" to the "autonomous future", users were no longer focused on simple

plugin/IDE improvements. The three dominant demands are for the AI to evolve from an assistant into an architect, an agent, and an expert. Multimodality the next most required feature.

Architect (Reasoning). The top request is for an AI that can perform holistic, multi-file architectural design and create complex projects from scratch. High level specifications to full functioning code is also expected, as well as sufficient creative prowess to generate novel algorithms.

Agent (Autonomy). The second-largest theme is a desire for a fully autonomous software engineering agent that can manage the entire life-cycle, from running experiments to making pull requests and monitoring deployments, not only fix issues.

Expert (Context and Compliance). Finally, users want the AI to have deep understanding of two areas. (a) Full repository understanding. This mirrors a request from short term improvements, underlying how important it is for practitioners. (b) Rules. Specifically, awareness of security, compliance, and governance across geographical regions.

Customization emerged as a strong theme for the ideal AI coding assistant. When asked how important for them is the ability to customize the behavior of AI coding assistant, 50% responded “Somewhat important” and 30% “Extremely important”. In terms of how, coding style, naming conventions, domain-specific knowledge (including best practices / security / compliance), ever-evolving public libraries have been mentioned.

Future AI Coding Assistant Takeaway

Future AI coding assistants will require

- (1) full customization of models and coding style.
- (2) optimal context: no size limitation, full repository/project understanding.
- (3) ability to handle complexity in terms of reasoning, compliance and autonomy in performing tasks for the entire software development life-cycle.

5 Conclusion

In this paper we presented the results from empirical study of 57 users spanning across multiple enterprise functions (e.g. finance, research, etc.) about their usage and experience with AI coding assistants. Our key takeaways include that the custom CodeLLM models within AI coding assistants are desirable, optimized and targeted for specific use cases. We reaffirmed that the AI coding assistants are helping drive the productivity for the users, irrespective of their programming skill level. To complement our study results, we analyzed additional 35 user surveys, to include academic point of view as a comparison to our enterprise-focused study. As AI coding assistants are evolving rapidly, most promise a solution for delivering features more quickly, and reduce work on repetitive tasks. In our future work will tackle the study and understanding of the agentic AI coding solutions and how they change the software development life-cycle.

References

- [1] Magnus Chukwuebuka Ahuchogu, Pravin Ganpatrao Gawande, Charu Mohla, Deepak A. Vidhate, Nidal Al Said, and Eric Howard. 2025. Evaluating the Impact of Generative AI on Intelligent Programming Assistance and Code Quality. *Power System Technology* 49, 1 (2025), 5–10.
- [2] Mehmet Akhoroz and Caglar Yildirim. 2025. Conversational AI as a Coding Assistant: Understanding Programmers’ Interactions with and Expectations from Large Language Models for Coding. arXiv:2503.16508 [cs.HC] <https://arxiv.org/abs/2503.16508>
- [3] Gal Bakal, Ali Dasdan, Yaniv Katz, Michael Kaufman, and Guy Levin. 2025. Experience with GitHub Copilot for Developer Productivity at Zoominfo. arXiv:2501.13282 [cs.SE] <https://arxiv.org/abs/2501.13282>
- [4] Leonardo Banh, Florian Holldack, and Gero Strobel. 2025. Copiloting the future: How generative AI transforms Software Engineering. *Inf. Softw. Technol.* 183, C (jun 2025), 14 pages. doi:10.1016/j.infsof.2025.107751
- [5] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv:2507.09089 [cs.AI] <https://arxiv.org/abs/2507.09089>
- [6] Neil C. C. Brown, Pierre Weill-Tessier, Juho Leinonen, Paul Denny, and Michael Kölling. 2025. Howzat? Appealing to Expert Judgement for Evaluating Human and AI Next-Step Hints for Novice Programmers. *ACM Transactions on Computing Education* 25, 3 (jul 2025), 1–43. doi:10.1145/3737885
- [7] Jenna Butler, Jina Suh, Sankeerti Haniyur, and Constance Hadley. 2024. Dear Diary: A randomized controlled trial of Generative AI coding tools in the workplace. arXiv:2410.18334 [cs.SE] <https://arxiv.org/abs/2410.18334>
- [8] Vincenzo Corso, Leonardo Mariani, Daniela Micucci, and Oliviero Riganelli. 2024. Generating Java Methods: An Empirical Assessment of Four AI-Based Code Assistants. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (Lisbon, Portugal) (ICPC ’24)*. Association for Computing Machinery, New York, NY, USA, 13–23. doi:10.1145/3643916.3644402
- [9] Kevin Crowston and Francesco Bolici. 2025. Deskilling and upskilling with AI systems. *Information Research an international electronic journal* 30, iConf (Mar. 2025), 1009–1023. doi:10.47989/ir30iConf47143
- [10] Kevin Zheyuan Cui, Mert Demirel, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. 2024. The Productivity Effects of Generative AI: Evidence from a Field Experiment with GitHub Copilot. *An MIT Exploration of Generative AI* (mar 27 2024). <https://mit-genai.pubpub.org/pub/v5iixksv>.
- [11] Zheyuan Cui, Mert Demirel, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. 2025. The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. doi:10.2139/ssrn.4945566
- [12] Joy Krishan Das, Saikat Mondal, and Chanchal K. Roy. 2024. Why Do Developers Engage with ChatGPT in Issue-Tracker? Investigating Usage and Reliance on ChatGPT-Generated Code. arXiv:2412.06757 [cs.SE] <https://arxiv.org/abs/2412.06757>
- [13] A. Dias Bahmed. 2024. Integrated development environments: Exploring the impact of the implementation of artificial intelligence on workflow efficiency and its potential for developer displacement.
- [14] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There’s more to it than you think. *Queue* 19, 1 (2021), 20–48. doi:10.1145/3454122.3454124
- [15] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Trans. Softw. Eng. Methodol.* (feb 2025). doi:10.1145/3716848 Just Accepted.
- [16] Leonardo Gambacorta, Han Qiu, Shuo Shan, and Daniel M Rees. 2024. *Generative AI and labour productivity: a field experiment on coding*. Vol. 1208. Bank for International Settlements, Monetary and Economic Department.
- [17] Alex Gu, Naman Jain, Wen-Ding Li, Manish Shetty, Kevin Ellis, Koushik Sen, and Armando Solar-Lezama. 2025. Tasks, Challenges, and Paths Towards AI for Software Engineering. In *ICLR 2025 Third Workshop on Deep Learning for Code*. <https://openreview.net/forum?id=t0QaoJqjdW>
- [18] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. 34, 5, Article 124 (may 2025), 30 pages. doi:10.1145/3712003
- [19] Victoria Jackson, Bogdan Vasilescu, Daniel Russo, Paul Ralph, Rafael Prikladnicki, Maliheh Izadi, Sarah D’Angelo, Sarah Inman, Anielle Andrade, and André van der Hoek. 2025. The Impact of Generative AI on Creativity in Software Development: A Research Agenda. 34, 5, Article 133 (may 2025), 28 pages. doi:10.1145/3708523
- [20] Jan H. Klemmer, Yasemin Acar, Laura Dietz, Daniel Zoller, Alexander Pretschner, and Simon Fenz. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the 2024 ACM Conference on Computer and Communications Security (CCS ’24)*. Association for Computing Machinery, New York, NY, USA, 2726–2740. doi:10.1145/3658644.3690283
- [21] Mohammad Amin Kuhail, Sujith Samuel Mathew, Ashraf Khalil, Jose Berengueres, and Syed Jawad Hussain Shah. 2024. “Will I be replaced?” Assessing ChatGPT’s effect on software development and programmer perceptions of AI tools. *Sci. Comput. Program.* 235, C (jul 2024), 21 pages. doi:10.1016/j.scico.2024.103111
- [22] Michael Jing Long Lee, Advait Sarkar, and Alan F. Blackwell. 2024. Predictability of identifier naming with Copilot: A case study for mixed-initiative programming tools. In *Proceedings of the 35th Annual Conference of the Psychology of Programming Interest Group (PPIG 2024)*. <https://www.microsoft.com/en->

- us/research/publication/predictability-of-identifier-naming-with-copilot-a-case-study-for-mixed-initiative-programming-tools/
- [23] Jia Li et al. 2025. Assessing open source software developer's opportunity costs in a time of generative AI. *SSRN Electronic Journal* (2025). doi:sol3/papers.cfm?abstract_id=5100609
 - [24] Jie Li, Youyang Hou, Laura Lin, Ruihao Zhu, Hancheng Cao, and Abdallah El Ali. 2025. Vibe Coding for UX Design: Understanding UX Professionals' Perceptions of AI-Assisted Design and Development. *arXiv:2509.10652 [cs.HC]* <https://arxiv.org/abs/2509.10652>
 - [25] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 52, 13 pages. doi:10.1145/3597503.3608128
 - [26] Joe Llerena-Izquierdo, Ingrid Fiallos-Vargas, Jonatan Portugal-Gorozabel, and Alonso Veloz-Arce. 2024. Towards an experience in AI-driven development for programming applied to multimedia using Tabnine. In *2024 IEEE URUCON*. 1–5. doi:10.1109/URUCON63440.2024.10850207
 - [27] Yunbo Lyu, Zhou Yang, Jieke Shi, Jianming Chang, Yue Liu, and David Lo. 2025. "My productivity is boosted, but ..." Demystifying Users' Perception on AI Coding Assistants. *arXiv:2508.12285 [cs.SE]* <https://arxiv.org/abs/2508.12285>
 - [28] Alina Mailach, Dominik Gorgosch, Norbert Siegmund, and Janet Siegmund. 2024. "Ok Pal, we have to code that now": interaction patterns of programming beginners with a conversational chatbot. *Empirical Software Engineering* 30, 1 (2024), 1–35. doi:10.1007/s10664-024-10561-6
 - [29] Boris Martinović and Robert Rozić. 2025. Perceived Impact of AI-Based Tooling on Software Development Code Quality. *SN Computer Science* 6, 1 (2025), 1–14. doi:10.1007/s42979-024-03608-4
 - [30] McKinsey Digital. 2023. Unleashing developer productivity with generative AI. *McKinsey and Company* (2023). <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/unleashing-developer-productivity-with-generative-ai>
 - [31] Courtney Miller, Rudrajit Choudhuri, Mara Ulloa, Sankeerti Haniyur, Robert DeLine, Margaret-Anne Storey, Emerson Murphy-Hill, Christian Bird, and Jenna L. Butler. 2025. "Maybe We Need Some More Examples:" Individual and Team Drivers of Developer GenAI Tool Use. *arXiv:2507.21280 [cs.SE]* <https://arxiv.org/abs/2507.21280>
 - [32] Amr Mohamed, Maram Assi, and Mariam Guizani. 2025. The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review. *arXiv:2507.03156 [cs.SE]* <https://arxiv.org/abs/2507.03156>
 - [33] Stoyan Nikolov, Daniele Codecasa, Anna Sjövall, Maxim Tabachnyk, Satish Chandra, Siddharth Taneja, and Celal Ziftci. 2025. How is Google using AI for internal code migrations? *arXiv:2501.06972 [cs.SE]* <https://arxiv.org/abs/2501.06972>
 - [34] Gabrielle O'Brien. 2025. How Scientists Use Large Language Models to Program. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 876, 16 pages. doi:10.1145/3706598.3713668
 - [35] Behrooz Omidvar Tehrani, Ishaani M, and Anmol Anubhai. 2024. Evaluating Human-AI Partnership for LLM-based Code Migration. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI EA '24)*. Association for Computing Machinery, New York, NY, USA, Article 133, 8 pages. doi:10.1145/3613905.3650896
 - [36] Stack Overflow. 2024. 2024 Developer Survey: Technology - Integrated Development Environment. <https://survey.stackoverflow.co/2024/technology#1-integrated-development-environment>
 - [37] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2025. How Much Does AI Impact Development Speed? an Enterprise-Based Randomized Controlled Trial. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 618–629. doi:10.1109/ICSE-SEIP66354.2025.00060
 - [38] Guilherme Vaz Pereira, Victoria Jackson, Rafael Prikladnicki, André van der Hoek, Luciane Fortes, Carolina Araújo, André Coelho, Ligia Chelli, and Diego Ramos. 2025. Exploring GenAI in Software Development: Insights from a Case Study in a Large Brazilian Company. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 330–341. doi:10.1109/ICSE-SEIP66354.2025.00035
 - [39] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. ACM, 2785–2799. doi:10.1145/3576915.3623157
 - [40] James Prather et al. 2024. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research (ICER '24)*. Association for Computing Machinery, New York, NY, USA, 302–311. doi:10.1145/3639535.3650630
 - [41] Sadra Sabouri, Philipp Eibl, Xinyi Zhou, Morteza Ziyadi, Nenad Medvidovic, Lars Lindemann, and Souti Chattopadhyay. 2025. Trust Dynamics in AI-Assisted Development: Definitions, Factors, and Implications. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1678–1690. doi:10.1109/ICSE55347.2025.00199
 - [42] Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (Feb. 2025), 107610. doi:10.1016/j.infsof.2024.107610
 - [43] Anshul Shah, Anya Chernova, Elena Tomson, Leo Porter, William G. Griswold, and Adalbert Gerald Soosai Raj. 2025. Students' Use of GitHub Copilot for Working with Large Code Bases. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (Pittsburgh, PA, USA) (SIGSEST 2025)*. Association for Computing Machinery, New York, NY, USA, 1050–1056. doi:10.1145/3641554.3701800
 - [44] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Laura A. Heymann, Massimiliano Di Penta, Daniel M German, and Denys Poshyvanyk. 2025. Developer Perspectives on Licensing and Copyright Issues Arising from Generative AI for Software Development. *ACM Transactions on Software Engineering and Methodology* (jun 2025). doi:10.1145/3743133
 - [45] Viktoriya Stray, Nils Brede Moe, Nivethika Ganeshan, and Simon Kobbenes. 2025. Generative AI and Developer Workflows: How GitHub Copilot and ChatGPT Influence Solo and Pair Programming. In *Proceedings of the 58th Hawaii International Conference on System Sciences (HICSS)*. 1–10. doi:10.24251/HICSS.2025.883
 - [46] Wannita Takerngsaksiri, Chakkrit Tantithamthavorn, Micheal Fu, Jirat Pasuksmit, Kun Chen, and Ming Wu. 2025. Code Readability in the Age of Large Language Models: An Industrial Case Study from Atlassian. *arXiv:2501.11264 [cs.SE]* <https://arxiv.org/abs/2501.11264>
 - [47] Valerio Terragni, Annie Vella, Partha Roop, and Kelly Blincoe. 2025. The Future of AI-Driven Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 120 (May 2025), 20 pages. doi:10.1145/3715003
 - [48] Norbert Tihanyi, Tamas Bisztray, Mohamed Amine Ferrag, Ridhi Jain, and Lucas C. Cordeiro. 2024. How secure is AI-generated code: a large-scale comparison of large language models. *Empirical Software Engineering* 30, 2 (2024), 1–29. doi:10.1007/s10664-024-10590-1
 - [49] Thiago S. Vaillant, Felipe Devesa de Almeida, Paulo Anselmo M. S. Neto, Cuiyun Gao, Jan Bosch, and Eduardo Santana de Almeida. 2024. Developers' Perceptions on the Impact of ChatGPT in Software Development: A Survey. *arXiv:2405.12195 [cs.SE]* <https://arxiv.org/abs/2405.12195>
 - [50] Wei Wang, Huilong Ning, Gaowei Zhang, Libo Liu, and Yi Wang. 2024. Rocks Coding, Not Development: A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks. *Proc. ACM Softw. Eng.* 1, FSE, Article 32 (jul 2024), 23 pages. doi:10.1145/3643758
 - [51] Thomas Weber, Maximilian Brandmaier, Albrecht Schmidt, and Sven Mayer. 2024. Significant Productivity Gains through Programming with Large Language Models. 8, EICS, Article 256 (jun 2024), 29 pages. doi:10.1145/3661145
 - [52] Justin D. Weisz, Shraddha Kumar, Michael Muller, Karen-Ellen Browne, Arielle Goldberg, Ellice Heintze, and Shagun Bajpai. 2025. Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise. *arXiv:2412.06603 [cs.HC]* <https://arxiv.org/abs/2412.06603>
 - [53] Xiyu Zhou, Peng Liang, Beiqi Zhang, Zengyang Li, Aakash Ahmad, Mojtaba Shahin, and Muhammad Waseem. 2025. Exploring the problems, their causes and solutions of AI pair programming: A study on GitHub and Stack Overflow. *J. Syst. Softw.* 219, C (jan 2025), 17 pages. doi:10.1016/j.jss.2024.112204
 - [54] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2024. Measuring GitHub Copilot's Impact on Productivity. *Commun. ACM* 67, 3 (feb 2024), 54–63. doi:10.1145/3633453